Smart Billing & Inventory Management System 4 Python Project

A Python-based billing system using Pandas and CSV that supports customer-type discounts, GST calculation (CGST+SGST), stock updates, shortage logging, and invoice printing.

Team No. 1

Presented By:

- **Pranay Singh -** Roll No. **02**
- **Darshan Fanse** - Roll No. **07**
- **Aryan Sinha -** Roll No. **36**
- **Parth Jibhakate -** Roll No. **47**
- **Yashodeep Dube -** Roll No. **01**

Project Overview

Problem Statement & Objective

Problem Statement

The need for a robust, automated billing system that accurately calculates taxes (GST), applies customer-specific discounts, and manages inventory in real-time to prevent stockouts and track shortages.

Objective

1. **Accurate Billing**

   Implement GST calculation (CGST+SGST) and customer-type discounts.

2. **Real-time Inventory**

   Update stock quantities immediately after a sale.

3. **Shortage Logging**

   Automatically log stock shortages for restocking purposes.

4. **Professional Output**

   Generate a clean, printable bill/invoice.

Key Features & Technologies

Python

Core programming language for logic and execution.

Pandas

Used for efficient data handling and manipulation of CSV files.

CSV Files

Lightweight, file-based storage for products, customers, and stock logs.

Discounts & GST

Supports customer-type discounts and CGST+SGST calculation.

Data Structure

Dataset Description

The system relies on two primary CSV files for data storage: `products.csv` and `customers.csv`.

products.csv (Inventory)

Item ID	ItemName	Price	TaxRate	StockQty
101	Pen	10	0.18	100
102	Notebook	50	0.12	50
103	Eraser	5	0.05	200
104	Marker	25	0.18	80

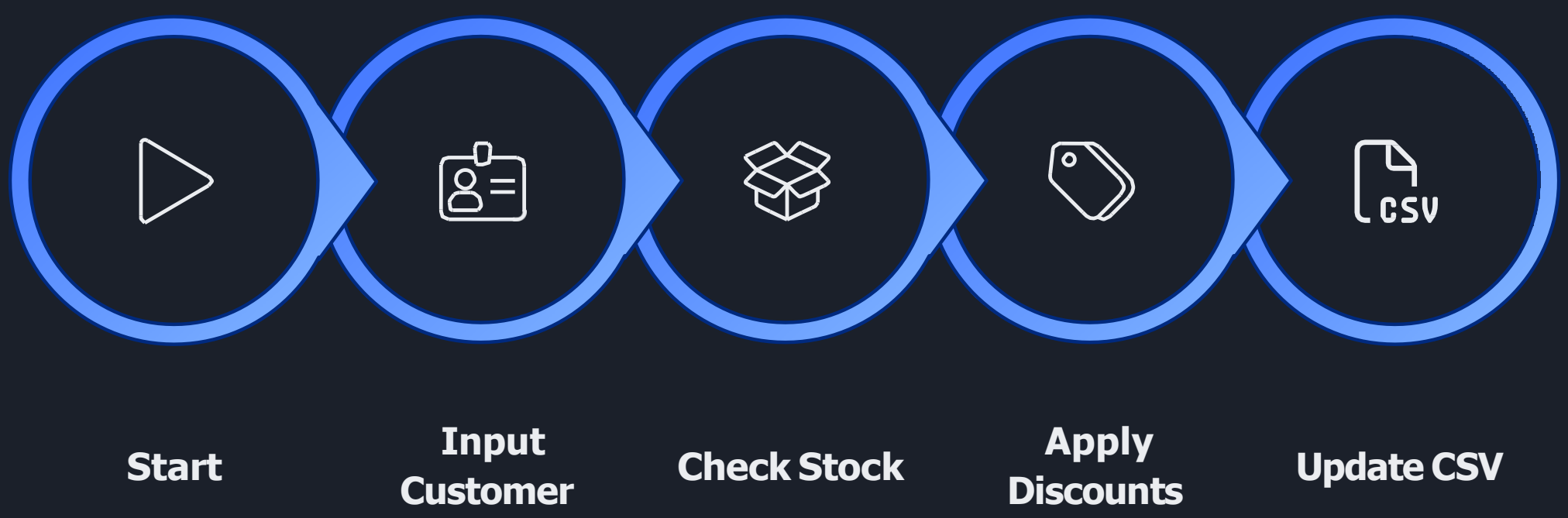
customers.csv (Discounts)

CustomerID	Name	Type
C001	Alice	Regular
C002	Bob	Premium
C003	Charlie	Regular

Discount Rates:

- Regular: 5%
- Premium: 10%

Billing Flowchart



Implementation

Full Python Code: billing_cli.py (Part 1 of 2)

```
import pandas as pd
from datetime import datetime

# Load data
products = pd.read_csv("products.csv")
customers = pd.read_csv("customers.csv")

DISCOUNTS = {'Regular': 0.05, 'Premium': 0.10}


def log_shortage(item, requested, available):
    df = pd.DataFrame([{
        "Date": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "Item": item,
        "Requested": requested,
        "Available": available,
        "Shortage": requested - available
    }])

    try:
        old = pd.read_csv("stock_log.csv")
        df = pd.concat([old, df], ignore_index=True)
    except:
        pass

    df.to_csv("stock_log.csv", index=False)


def generate_bill(customer_id, cart):
    customer = customers[customers.CustomerID == customer_id].iloc[0]
    name, ctype = customer["Name"], customer["Type"]
    discount_rate = DISCOUNTS.get(ctype, 0)

    bill_items = []
    total = 0

    for item_id, qty in cart:
        product = products[products.ItemID == item_id].iloc[0]
        item, price, stock = product["ItemName"], product["Price"], product["StockQty"]
```

Implementation

Full Python Code: billing_cli.py (Part 2 of 2)

```
if stock == 0:
        print(f"

{item} OUT OF STOCK - Skipped")
        log_shortage(item, qty, 0)
        continue

    if qty > stock:
        print(f" Only {stock} {item} available. Billing partial.")
        log_shortage(item, qty, stock)
        qty = stock

    subtotal = price * qty
    total += subtotal

    products.loc[products.ItemID == item_id, "StockQty"] -= qty
    bill_items.append([item, qty, price, subtotal])

discount = total * discount_rate
taxable = total - discount
avg_tax = products.loc[products.ItemID.isin([x[0] for x in cart]), "TaxRate"].mean()
tax = taxable * avg_tax
final = taxable + tax

print("\n BILL RECEIPT")
print(f"Customer: {name} ({ctype})")
print("--------------------------")
for item, q, p, s in bill_items:
    print(f"{item} x{q} @ ¹{p} = ¹{s}")
print("--------------------------")
print(f"Subtotal: ¹{total:.2f}")
print(f"Discount: -¹{discount:.2f}")
print(f"Tax: ¹{tax:.2f}")
print(f"Total Payable: ¹{final:.2f}")
print(" Purchase Successful!\n")

products.to_csv("products.csv", index=False)

def menu():
    while True:
        print("\n=== Billing System ===")
        print("1. Show Products")
        print("2. Generate Bill")
        print("3. Exit")
        ch = input("Enter choice: ")

        if ch == "1":
            print(products)
        elif ch == "2":
            cid = input("Enter Customer ID: ")
            cart = []
            while True:
                item = int(input("Item ID: "))
                qty = int(input("Qty: "))
                cart.append((item, qty))
                if input("Add more? (y/n): ") == "n":
                    break
            generate_bill(cid, cart)
        elif ch == "3":
            print("Exiting...")
            break
        else:
            print("Invalid choice")

menu()
```

How to Run the System

01

Setup Files

Put products.csv and customers.csv in the same folder as billing_cli.py.

02

Install Dependencies

pip install pandas python-pptx

03

Run CLI

python billing_cli.py

04

Auto-Create Presentation (Optional)

python generate_ppt.py

05

Verify Updates

After a billing session check products.csv to see updated StockQty and stock_log.csv for shortages.

Implementation

Sample Run: Console Interaction

Here's an example of how a user would interact with the `billing_cli.py` system from the command line, demonstrating product listing, bill generation, and inventory updates.

Showing Products & Initial Menu

```
=== Billing System ===
1. Show Products
2. Generate Bill
3. Exit
Enter choice: 1

  ItemID  ItemName  Price  TaxRate  StockQty
0    101       Pen     10     0.18       100
1    102  Notebook     50     0.12        50
2    103    Eraser      5     0.05       200
3    104    Marker     25     0.18        80

=== Billing System ===
1. Show Products
2. Generate Bill
3. Exit
Enter choice: 2
```

Generating a Bill & Shortage Log

```
Enter Customer ID: C002
Item ID: 101
Qty: 5
Add more? (y/n): y
Item ID: 102
Qty: 60
Only 50 Notebook available. Billing partial.
Add more? (y/n): n

 BILL RECEIPT
Customer: Bob (Premium)
--------------------
Pen x5 @¹10 = ¹50.0
Notebook x50 @¹50 = ¹2500.0
---------------------------
Subtotal: ¹2550.00
Discount: -¹255.00
Tax: ¹412.35
Total Payable: ¹2707.35
 Purchase Successful!

=== Billing System ===
1. Show Products
2. Generate Bill
3. Exit
Enter choice: 3
Exiting...
```

Observe the partial billing for "Notebook" and the automatic calculation of discount and tax, leading to the final payable amount. Post-transaction, the system updates `products.csv` for stock changes and `stock_log.csv` for any shortages.

Future Scope: Evolving the System

Our billing and inventory management system offers a strong foundation, with significant potential for future enhancements to boost functionality and user experience.

Enhanced User Interface (GUI)

Integrate a GUI using Tkinter or Streamlit for intuitive navigation and improved usability, making the system more accessible to all users.

Robust Database Integration

Transition from CSV to MySQL/SQLite for scalable data storage, ensuring better performance, data integrity, and multi-user access.

Data Analytics & Visualization

Implement dynamic dashboards to visualize sales trends, stock movement, and customer purchasing patterns for informed strategic decision-making.

Cloud-Based Inventory Updates

Enable real-time inventory updates and synchronization across multiple stores or access points via secure cloud integration for seamless operations.

Digital Invoice Delivery

Add features to generate and send digital invoices directly to customers via email or popular messaging platforms like WhatsApp for convenience.

References

This project leveraged various trusted resources to ensure robust design and accurate implementation.

- Python Documentation
- Pandas Library Documentation
- CSV Module Guide (GeeksforGeeks)
- TutorialsPoint 3 Python File Handling
- Project-inspired references: Various online resources on Python-based billing and inventory systems for conceptual understanding and best practices.

Conclusion: Empowering Business Efficiency

The Smart Billing & Inventory Management System provides a robust and automated solution for retail operations, ensuring accuracy and optimizing stock control. This project demonstrates how Python can be leveraged to build practical tools that streamline daily business processes.

Real-time Inventory

Minimizes stockouts and provides accurate product availability, enhancing customer satisfaction.

Automated Billing

Reduces manual errors with built-in discount application and precise tax calculations.

Data-Driven Insights

Comprehensive logging of sales and shortages supports informed decision-making and inventory optimization.